Implantação numa VM do Google (Compute Engine)

Documentação técnica - Prumo

> **Situação:** os scripts de `deploy/vm/` foram testados num Ubuntu 24.04 limpo (instalação, banco, nginx, envio de vídeo de 150 MB, backup e restauração). A VM e o certificado HTTPS real ainda não foram criados. Preços e limites do Google mudam: confira na calculadora antes de criar a VM.

Tudo roda numa única máquina: PostgreSQL, API, frontend e os arquivos (PDFs e vídeos). É o caminho mais simples para o piloto: um servidor, um custo fixo e nenhum serviço externo além do Google.

1. Arquitetura

```
Navegador -> https://<domínio> -> nginx (portas 80/443, certificado Let's Encrypt)
                                  +- /            frontend/dist (arquivos estáticos)
                                  +- /uploads/    backend/uploads (PDFs e vídeos, direto do disco)
                                  +- /api/        API Node em 127.0.0.1:4000 (serviço systemd)
                                                    +- PostgreSQL 16 em 127.0.0.1:5432
```

Peça	Onde fica na VM
Código	`/opt/manual-operacional` (cópia do repositório)
Configuração e senhas	`/opt/manual-operacional/backend/.env` (gerado pelo instalador, só o usuário `manual` lê)
Banco	PostgreSQL local, base `manual_operacional`, usuário `manual_app`. Porta 5432 fechada para fora
PDFs e vídeos	`/opt/manual-operacional/backend/uploads`
API	serviço `manual-operacional-api`, roda como usuário `manual`
Backups do banco	`/var/backups/manual-operacional` (diário, 14 dias)

2. Criar a VM

No console do Google Cloud: **Compute Engine > Instâncias de VM > Criar instância**.

Campo	Valor recomendado
Nome	`manual-operacional`
Região	`southamerica-east1` (São Paulo): os dados ficam no Brasil (LGPD)
Tipo de máquina	`e2-small` (2 vCPU compartilhadas, 2 GB)
Disco de inicialização	Ubuntu 24.04 LTS, 30 GB, disco equilibrado
Firewall	marque **Permitir tráfego HTTP** e **Permitir tráfego HTTPS**

- **Opção sem custo:** a `e2-micro` com 30 GB de disco padrão entra no nível gratuito do Google, mas só nas regiões `us-east1`, `us-central1` e `us-west1` (dados nos EUA) e tem 1 GB de memória. O instalador cria 2 GB de memória de troca, o que basta para o piloto.
- **IP fixo:** em **Rede VPC > Endereços IP**, promova o IP externo da VM para **estático**. Sem isso, o IP muda quando a VM reinicia e o endereço do sistema para de funcionar.

3. Escolher o endereço

O HTTPS precisa de um nome, não só do IP.

- **Sem domínio próprio:** use o sslip.io, gratuito e sem cadastro. Para o IP `34.95.10.20`, o endereço é `34-95-10-20.sslip.io`.
- **Com domínio do hospital:** peça à TI um registro DNS tipo **A**, por exemplo `manual.plenasaude.com.br` apontando para o IP da VM. Dá para começar com sslip.io e trocar depois rodando o instalador de novo com o novo nome.

4. Copiar o código para a VM

Abra o terminal da VM pelo botão **SSH** no console e rode:

```
sudo git clone https://github.com/<organizacao>/<repositorio>.git /opt/manual-operacional
```

- **Repositório privado:** crie no GitHub um *fine-grained token* só com **Contents: Read** neste repositório e use `https://<TOKEN>@github.com/<organizacao>/<repositorio>.git`. O token fica gravado em `/opt/manual-operacional/.git/config`; revogue-o no GitHub se a VM for desativada.
- Não copie o `backend/.env` do seu computador. O instalador gera um novo, com senhas próprias, e para com aviso se encontrar um `.env` apontando para outro banco.

5. Instalar

```
cd /opt/manual-operacional
sudo bash deploy/vm/instalar.sh 34-95-10-20.sslip.io seu-email@plenasaude.com.br
```

O script faz, em ordem:

1. Instala PostgreSQL, nginx, certbot e Node.js 22.
2. Cria 2 GB de memória de troca se a VM tiver menos de 3 GB.
3. Cria o usuário de sistema `manual`, dono do código e dos arquivos.
4. Cria o banco `manual_operacional` e o usuário `manual_app` com senha aleatória, roda `backend/sql/schema.sql` e grava o `backend/.env`.
5. Instala as dependências e gera o frontend apontando para `https://<domínio>`.
6. Cria e inicia o serviço `manual-operacional-api`, que reinicia sozinho em caso de falha e ao religar a VM.
7. Configura o nginx (vídeos até 200 MB, cache dos arquivos estáticos) e emite o certificado HTTPS. A renovação é automática.
8. Agenda a cópia diária do banco.

Ao final aparece `Pronto: https://<domínio>`. Entre com **admin / admin123** e troque a senha na hora.

O script pode ser rodado de novo, por exemplo para trocar o domínio: não apaga dados nem troca as senhas já geradas.

6. Publicar uma nova versão

1. No seu computador, faça commit e push no VS Code.
2. Na VM:

```
sudo bash /opt/manual-operacional/deploy/vm/atualizar.sh
```

O script baixa o código, reinstala dependências, gera o frontend, reinicia a API e confere `/health/ready`.

Mudança no banco: o `atualizar.sh` não altera tabelas. Quando `backend/sql/schema.sql` mudar, rode antes, como o usuário da aplicação (assim as tabelas novas ficam com o dono certo):

```
cd /opt/manual-operacional
sudo bash /etc/cron.daily/manual-operacional-backup
psql "$(sudo grep '^DATABASE_URL=' backend/.env | cut -d= -f2-)" -v ON_ERROR_STOP=1 -f backend/sql/schema
.sql
```

7. Backup

O quê	Como
Banco	`pg_dump` diário em `/var/backups/manual-operacional`, guarda 14 dias (`deploy/vm/backup.sh`)
PDFs, vídeos e os próprios backups	**Programação de snapshot** do disco: Compute Engine > Snapshots > Criar programação, diária, retenção de 7 dias, e anexe ao disco da VM

O backup diário fica no mesmo disco da VM: sem o snapshot, perder a VM é perder tudo.

Restaurar o banco (com a API parada):

```
sudo systemctl stop manual-operacional-api
sudo -u postgres pg_restore --clean --if-exists -d manual_operacional < /var/backups/manual-operacional/b
anco-AAAA-MM-DD.dump
sudo systemctl start manual-operacional-api
```

8. Operação do dia a dia

Preciso…	Comando na VM
Ver se a API está rodando	`systemctl status manual-operacional-api`
Ver os erros da API	`journalctl -u manual-operacional-api -n 100`
Reiniciar a API	`sudo systemctl restart manual-operacional-api`
Conferir banco e API	`curl http://127.0.0.1:4000/health/ready`
Abrir o banco no terminal	`sudo -u postgres psql manual_operacional`
Ver espaço em disco	`df -h /` e `du -sh /opt/manual-operacional/backend/uploads`

pgAdmin no seu computador: a porta do banco não fica aberta na internet. Use um túnel SSH (requer o Google Cloud CLI instalado):

```
gcloud compute ssh manual-operacional --zone=<zona> -- -L 5433:127.0.0.1:5432
```

Com o túnel aberto, conecte o pgAdmin em `localhost`, porta `5433`, base `manual_operacional`, usuário `manual_app` e a senha que está no `DATABASE_URL` do `.env` da VM.

9. Limitações conhecidas

- **Uma máquina só:** se a VM parar, o sistema fica fora até ela voltar. Aceitável para o piloto.
- **Links dos arquivos não expiram:** PDFs e vídeos ficam em `/uploads/` com nomes aleatórios. Quem tiver o link abre o arquivo sem login. Para produção, use um bucket (R2, S3 ou Cloud Storage) no modo `s3`, que gera links temporários. Basta preencher as variáveis `S3_*`.
- **Espaço em disco:** os vídeos ocupam o disco da VM. Com 30 GB cabem cerca de 100 vídeos de 200 MB. O disco pode ser aumentado no console sem recriar a VM.

10. Migrar depois do piloto

- **Banco:** `pg_dump` na VM e restauração no novo PostgreSQL (Cloud SQL ou outro).
- **Arquivos:** copie `backend/uploads` para o bucket mantendo os caminhos, ou mantenha a VM só para eles até recriar os manuais.